ارزیابی و کارایی سیستم ها

نام و نام خانوادگی: سید علی آقامیری

پروژه

شبیه سازی ۴.۷ :

در قسمت ۴.۷ کتاب $lawsimulationmodeling$به شبیه سازی یک سیمولیشن $randomnumbergenerator$ به زبان پایتون به صورت شی گرا میپردازیم. $Generators$ اعداد تصادفی، ابزارهایی هستند که برای تولید توزیع‌های تصادفی مختلف استفاده می‌شوند. این توزیع‌ها می‌توانند برای شبیه‌سازی فرآیندهای تصادفی در علوم مختلف، مدل‌سازی مالی، شبیه‌سازی‌های کامپیوتری و سایر کاربردها مفید باشند. دو نوع اصلی از $generators$ اعداد تصادفی وجود دارند:

$Generators$ با توزیع‌های ثابت: این $generators$ معمولاً از الگوریتم‌هایی استفاده می‌کنند که اعداد تصادفی را با توزیع یکنواخت تولید می‌کنند. به عنوان مثال، یک $generator$ می‌تواند اعدادی بین ۰ و ۱ تولید کند که به صورت یکنواخت توزیع شده‌اند. $Generators$ با توزیع‌های خاص: این $generators$ برای تولید اعداد با توزیع‌های خاص مانند نرمال، اکسپونانسیل، یا پواسون طراحی شده‌اند. اگر بخواهیم شرح مختصری از کد ارائه شده بدهیم به صورت زیر است : کلاس ‘$RandomNumberGenerator$’ در این کد به تولید اعداد تصادفی با استفاده از دو روش مختلف می‌پردازد: $LinearCongruentialGenerator(LCG)$ و $MultipleRecursiveGenerator$ (MRG). این دو روش از الگوریتم‌های تولید اعداد تصادفی هستند که در شبیه‌سازی‌ها و محاسبات تصادفی مختلف مورد استفاده قرار می‌گیرند.

$Init$: این تابع سازنده کلاس است که مقدارهای اولیه را برای تولید اعداد تصادفی تنظیم می‌کند. $Lcgrand$: یک عدد تصادفی در محدوده ‘[۰، ۱)’ برای یک جریان مشخص تولید می‌کند. $Lcgrandst$: مقدار اولیه ($seed$) را برای یک جریان خاص تنظیم می‌کند. $Lcgrandgt$: مقدار اولیه فعلی را برای یک جریان مشخص برمی‌گرداند. $Expon$: یک عدد تصادفی با توزیع نمایی و میانگین مشخص تولید می‌کند. $Mradst$: مقدار اولیه ($seed$) را برای یک جریان چندگانه ($multistream$) تنظیم می‌کند. $Mrand$: یک عدد تصادفی برای یک جریان چندگانه ($multistream$) تولید می‌کند. $Mrandgt$: مقدار اولیه فعلی را برای یک جریان چندگانه ($multistream$) برمی‌گرداند.

خروجی‌های این کد مقادیر تصادفی هستند که با استفاده از روش‌های مختلف تولید اعداد تصادفی به دست می‌آیند.

۱. : $rng.lcgrand$

این خروجی یک عدد تصادفی در محدوده ۰ و ۱ است که با استفاده از روش خطی-ضرب ($LinearCongruentialGenerator$) تولید شده است. مقدار خروجی نشان‌دهنده یک عدد تصادفی است که برای جریان شماره ۱ تولید شده است.

۲. $expon$:

این خروجی یک عدد تصادفی با توزیع نمایی ($ExponentialDistribution$) است که میانگین آن برابر ۰.۱ است. مقدار خروجی نشان‌دهنده زمانی است که بین رخدادهای تصادفی با توزیع نمایی با میانگین ۰.۱ اتفاق می‌افتد.

۳. $rng.mrand$

روجی یک عدد تصادفی برای جریان شماره ۱ است که با استفاده از روش چندگانه ($MultistreamRandomNumberGenerator$) تولید شده است. مقدار خروجی نشان‌دهنده یک عدد تصادفی در محدوده '[۰، ۱)' است که با استفاده از ترکیب دو رشته تصادفی مستقل تولید شده است.

شبیه سازی ۵.۲ :

مدل کامپیوتری به اشتراک گذاشته شده زمانی

یک سیستم کامپیوتری به اشتراک گذاشته شده زمانی نوعی از سیستم چند کاربره است که به کاربران متعدد اجازه می‌دهد به طور همزمان به یک سیستم کامپیوتری مرکزی دسترسی داشته باشند. ایده اصلی پشت به اشتراک گذاری زمانی این است که زمان CPU به قطعات کوچک (کوانتا زمانی) تقسیم شود و این قطعات به وظایف یا کاربران مختلف به صورت دورانی تخصیص داده شود. این کار باعث می‌شود که به هر کاربر این توهم داده شود که یک CPU اختصاصی دارد، در حالی که CPU واقعاً بین چندین کاربر به اشتراک گذاشته شده است.

توضیحات شبیه‌سازی

کد پایتون ارائه شده، یک مدل کامپیوتری به اشتراک گذاشته شده زمانی را شبیه‌سازی می‌کند. این شبیه‌سازی رفتار یک سیستم کامپیوتری را که در آن چندین کاربر (ترمینال‌ها) وظایف خود را ارسال می‌کنند و CPU این وظایف را طبق یک پروتکل اشتراک زمانی خاص پردازش می‌کند، ثبت می‌کند. در اینجا به نحوه انجام این کار توسط کد می‌پردازیم: ۱. مقداردهی اولیه و پارامترهای ورودی

شبیه‌سازی با مقداردهی اولیه مدل و خواندن پارامترهای ورودی از یک فایل ('$input25$'□□□). شروع می‌شود. این پارامترها شامل موارد زیر هستند:

- 'min_terms'، 'max_terms'، '$incr_terms$': حداقل، حداکثر و تعداد افزایشی ترمینال‌ها (کاربران) برای شبیه‌سازی. - '$num_responses_required$': تعداد وظایف لازم برای پردازش قبل از پایان شبیه‌سازی. - '$mean_think$': میانگین زمان فکر کردن بین وظایف ارسال شده توسط کاربران. - '$mean_service$': میانگین زمان سرویس برای هر وظیفه. - '$quantum$': کوانتا زمانی برای CPU. - '$swap$': زمان لازم برای تعویض وظایف در داخل و خارج از CPU.

۲. مدیریت رویدادها

شبیه‌سازی از یک روش شبیه‌سازی رویداد گسسته با انواع مختلف رویدادها استفاده می‌کند:

- '$EVENT_ARRIVAL$': ورود یک وظیفه جدید از یک ترمینال. - '$EVENT_END_CPU_RUN$': پایان یافتن کوانتا زمانی یک وظیفه در □□□. - '$EVENT_END_SIMULATION$': پایان شبیه‌سازی پس از پردازش تعداد

وظایف مورد نیاز. توابع اصلی مدیریت رویدادها عبارتند از: - ‘$arrive$()’: وظیفه ورود را مدیریت می‌کند، وظیفه را به صف اضافه می‌کند و اگر CPU بیکار است، آن را شروع می‌کند. - ‘end_CPU_run()’: پایان اجرای CPU را مدیریت می‌کند، یا وظیفه را دوباره به صف می‌برد یا آن را کامل می‌کند. - ‘$start_CPU_run$()’: شروع به اجرای یک وظیفه بر روی CPU، برنامه‌ریزی رویداد بعدی ‘end_CPU_run’. - ‘$timing$()’: مدیریت لیست رویدادها، پیشبرد ساعت شبیه‌سازی به رویداد بعدی. ۳. حلقه شبیه‌سازی
روش ‘$run_simulation$()’ از تعداد ترمینال‌های مختلف (از ‘min_terms’ تا ‘max_terms’ در گام‌های ‘$incr_terms$’) تکرار می‌کند. برای هر پیکربندی ترمینال‌ها، محیط شبیه‌سازی را مقداردهی اولیه می‌کند، وظایف اولیه ورود را برنامه‌ریزی می‌کند و رویدادها را تا زمانی که تعداد وظایف مورد نیاز پردازش شود، مدیریت می‌کند. ۴. گزارش‌دهی
در پایان شبیه‌سازی برای هر تعداد ترمینال، کد نتایج را به یک فایل خروجی (‘$output25.txt$’) می‌نویسد. گزارش شامل موارد زیر است: - تعداد ترمینال‌ها. - میانگین زمان پاسخ‌دهی (زمان لازم برای تکمیل یک وظیفه از زمان ارسال تا تکمیل). - میانگین تعداد وظایف در صف. - استفاده از CPU (بخش زمانی که CPU مشغول است). روش ‘$report$()’ مسئول تولید این خروجی‌ها است. خروجی‌های شبیه‌سازی فایل خروجی ‘$output25.txt$’ شامل نتایج شبیه‌سازی است که به صورت زیر ساختار یافته است: ۱. اطلاعات سرصفحه**: شامل جزئیات مربوط به پارامترهای شبیه‌سازی (تعداد ترمینال‌ها، میانگین زمان فکر کردن، میانگین زمان سرویس، کوانتا، زمان تعویض و تعداد وظایف پردازش شده). ۲. **نتایج شبیه‌سازی**: برای هر تعداد ترمینال، شامل: - تعداد ترمینال‌ها. - میانگین زمان پاسخ‌دهی. - میانگین تعداد وظایف در صف. - استفاده از CPU.

شبیه سازی ۴.۱ :

در قسمت ۴.۱ کتاب $lawsimulationmodeling$ به شبیه سازی $singleserverqueue$ میپردازیم (سیستم صف تک سرور را شبیه سازی میکنیم). باوجود سادگی این سیستم در مقایسه با سیستم های دیگر، نحوه شبیه سازی آن در واقع دارای پیچیدگی زیاد است. یک سیستم $SingleServerQueue$(صف تک‌خدمتی) مدلی از یک سیستم صف است که در آن مشتریان وارد صف می‌شوند و توسط یک سرور واحد خدمت‌رسانی می‌شوند. در این مدل، زمانی که یک مشتری وارد سیستم می‌شود، اگر سرور مشغول باشد، مشتری وارد صف انتظار می‌شود و زمانی که سرور آزاد شود، مشتری خدمت دریافت می‌کند. در کد پیاده سازی شده موارد زیر وجود دارد که به صورت کلی شرح داده میشود : خروجی این کد شامل گزارش‌هایی از عملکرد سیستم صف تک‌خدمتی است که شامل موارد زیر است: میانگین تأخیر در صف: میانگین زمانی که مشتریان در صف منتظر می‌مانند. میانگین تعداد افراد در صف: میانگین تعداد مشتریانی که در صف انتظار هستند. استفاده سرور: نسبت زمانی که سرور مشغول خدمت‌رسانی است به کل زمان شبیه‌سازی. زمان پایان شبیه‌سازی: زمان کلی که شبیه‌سازی اجرا شده است. ماژول های اصلی این کد عبارت است از موارد زیر : $_{init}$:

مقداردهی اولیه برای پارامترهای شبیه‌سازی مانند زمان‌های میانگین ورود و خدمت‌رسانی، تعداد مشتریان مورد نیاز، وضعیت اولیه سرور، تعداد مشتریان در صف و سایر متغیرهای مورد نیاز. $lcgrand$: تولید یک عدد تصادفی در محدوده [۰، ۱) با استفاده از روش $LinearCongruentialGenerator$. $initialize$: مقداردهی اولیه متغیرهای شبیه‌سازی مانند زمان شبیه‌سازی، وضعیت سرور، تعداد مشتریان در صف، زمان‌های ورود بعدی و زمان‌های خروج. $timing$: تعیین نوع رویداد بعدی (ورود یا خروج) و به‌روزرسانی زمان شبیه‌سازی. $arrive$: مدیریت رویداد ورود مشتری جدید. اگر سرور مشغول باشد، مشتری به صف اضافه می‌شود. در غیر این صورت، مشتری بلافاصله خدمت دریافت می‌کند. $depart$: مدیریت رویداد خروج مشتری. اگر صف خالی باشد، سرور آزاد می‌شود. در غیر این صورت، مشتری بعدی از صف خارج شده و خدمت‌رسانی به او آغاز می‌شود. $expon$: تولید زمان تصادفی با توزیع نمایی با میانگین مشخص برای ورود یا خدمت‌رسانی. $update_time_avg_stats$: به‌روزرسانی آمار زمانی میانگین صف و وضعیت سرور. $report$: تولید و نمایش گزارش نهایی شامل میانگین تأخیر در صف، میانگین تعداد افراد در صف، استفاده سرور و زمان پایان شبیه‌سازی. $run_simulation$: اجرای حلقه شبیه‌سازی تا زمانی که تعداد مشتریان خدمت‌رسانی شده به تعداد مورد نیاز برسد. در هر حلقه، نوع رویداد بعدی تعیین و رویداد مناسب اجرا می‌شود. $main$: تعریف پارامترهای شبیه‌سازی (میانگین زمان ورود، میانگین زمان خدمت‌رسانی و تعداد مشتریان مورد نیاز) و ایجاد و اجرای شیء شبیه‌سازی صف تک‌خدمتی. فایل ورودی $input14.txt$ باید شامل سه خط باشد که به ترتیب میانگین زمان ورود، میانگین زمان خدمت‌رسانی و تعداد مشتریان مورد نیاز را مشخص می‌کند. در تابع مقداردهی اولیه زمان اولین ورود، $timenextevent$ [۱]با افزودن یک متغیر تصادفی نمایی با ، ، تعیین می شود. تابع زمان $simtime$ $expon$($meaninterarrival$) fi $meaninterval$ برای مقایسه استفاده می شود $timenextevent$[۱] ، $timenextevent$[۲] ، . . . ، $timenextevent$[$numevents$] .) $numevents$ تابع اصلی تنظیم شده است(و $nexteventtype$ برابر با نوع رویدادی که زمان وقوع آن کوچکترین است تنظیم می‌کند. در صورت تساوی، نوع رویداد با کمترین شماره انتخاب می شود. سپس ساعت شبیه سازی به زمان وقوع نوع رویداد انتخابی، $mintimenextevent$ پیش می رود. برنامه کمی با بررسی خطا برای خالی بودن لیست رویداد پیچیده می شود، که به این معنی است که همه رویدادها در یک زمان برنامه ریزی شده اند. اگر چنین باشد)همانطور که توسط $nexteventtype$=۵ نشان داده شده است،(پیام خطا همراه با زمان ساعت فعلی)به عنوان یک کمک احتمالی برای رفع اشکال(تولید می شود و شبیه سازی خاتمه می یابد.

گزارش خروجی صف به صورت زیر میباشد :

$Single - serverqueueingsystem$

$Meaninterarrivaltime 1.000 minutes$

$Meanservicetime 0.500 minutes$

$Numberofcustomers 1000$

$Averagedelayinqueue 0.430 minutes$

$Averagenumberinqueue 0.418$

$Serverutilization 0.460$

$Timesimulationended 1027.915 minutes$

شبیه سازی ۵.۱ :

توضیحات عمومی

شبیه‌سازی سیستم موجودی یک روش مدیریت موجودی است که با استفاده از مدل‌های ریاضی و شبیه‌سازی کمک می‌کند تا عملکرد یک سیستم موجودی را تحلیل و بهینه‌سازی کنیم. در اینجا، یک کلاس به نام '$InventorySystem$' در پایتون تعریف شده است که ویژگی‌ها و رفتارهای یک سیستم موجودی را شبیه‌سازی می‌کند.

متغیرها و ورودی‌ها

۱. **متغیرهای ورودی**: - '$initial_inv_level$': سطح اولیه موجودی. - 'num_months': تعداد ماه‌های شبیه‌سازی. - '$num_policies$': تعداد سیاست‌های مختلف مدیریت موجودی. - 'num_values_demand': تعداد اندازه‌های مختلف تقاضا. - '$mean_interdemand$': میانگین زمان بین تقاضاها. - '$setup_cost$': هزینه برگزاری یک سفارش. - '$incremental_cost$': هزینه تکمیلی برای هر واحد سفارش. - '$holding_cost$': هزینه نگهداری هر واحد موجودی. - '$shortage_cost$': هزینه کمبود هر واحد موجودی. - '$minlag$': کمینه تاخیر تحویل سفارش. - '$maxlag$': بیشینه تاخیر تحویل سفارش.

۲. **متغیرهای دیگر**: - 'inv_level': سطح فعلی موجودی. - '$total_ordering_cost$': کل هزینه‌های سفارش‌دهی. - '$area_holding$': منطقه زیر نمودار هزینه نگهداری. - '$area_shortage$': منطقه زیر نمودار هزینه کمبود. - 'sim_time': زمان شبیه‌سازی فعلی. - '$time_last_event$': زمان آخرین رویداد. - '$time_next_event$': زمان رویدادهای بعدی. - '$prob_distrib_demand$': توزیع احتمال اندازه‌های تقاضا.

متدها و عملکرد

کلاس '$InventorySystem$' شامل مجموعه‌ای از متدها است که هر کدام وظایف خاصی در مدیریت موجودی دارند:

- **'$init$'**: مقادیر اولیه را از طریق ورودی‌ها دریافت و متغیرها را مقداردهی اولیه می‌کند. - **'$initialize$'**: وضعیت اولیه سیستم را بازنشانی می‌کند. - **'$expon$' و '$lcgrand$'**: توابع برای تولید اعداد تصادفی از توزیع

نمایی و توزیع خطی. - **‘$order_arrival$’ و :**‘□□□□□□□□’ مدیریت سفارش‌ها و ارزیابی نیاز به سفارش جدید. - **‘$update_time_avg_stats$’**: به‌روزرسانی آمارهای میانگین زمانی. - **‘$demand$’**: مدیریت تقاضاها. - **‘$random_integer$’**: تولید عدد تصادفی بر اساس توزیع احتمال تقاضا. - **‘$report$’**: گزارش نهایی از عملکرد سیستم موجودی. - **‘$simulate$’**: اجرای شبیه‌سازی بر اساس تعداد سیاست‌های مختلف مدیریت موجودی. - **‘$timing$’**: مدیریت زمان‌بندی رویدادها. - **‘$read_input_data$’ و ‘$write_output_data$’**: خواندن و نوشتن داده‌های ورودی و خروجی.